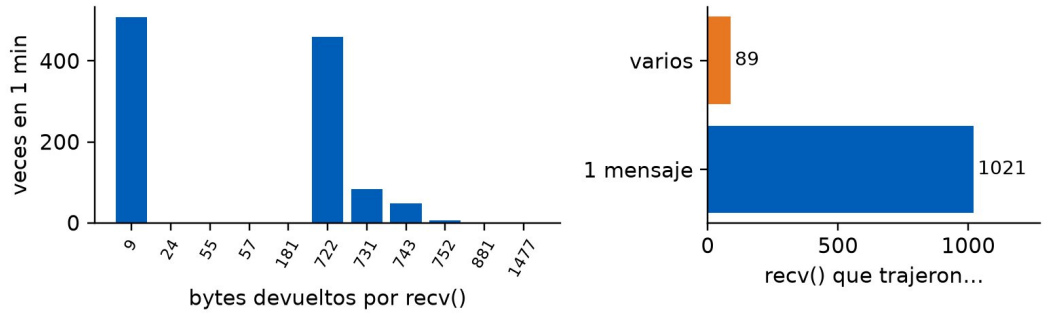
Preguntas de comprensión

Apartado 12 · respuestas con datos medidos en URSim

1. ¿Coincide cada recv() con un mensaje? ¿Qué consecuencia tiene?

No. En un minuto hubo 1110 `recv()` de 11 tamaños distintos (de 9 a 1477 bytes): 1021 trajeron un mensaje y 89 trajeron varios; también puede llegar uno a medias. Por eso nunca se procesa un `recv()` directamente: se acumula en un buffer y se corta por la longitud de cada mensaje.



2. ¿Por qué el mensaje y cada subpaquete empiezan con su longitud? ¿Y un tipo desconocido?

TCP es un flujo de bytes sin separadores: la longitud dice dónde acaba cada pieza. Un subpaquete de tipo desconocido se **salta** sumando su longitud, así el programa sigue funcionando aunque una versión nueva del robot añada datos.

3. ¿Qué pasa si se decodifica q_actual sin big-endian?

Salen valores absurdos porque los bytes se leen al revés. Ejemplo: −1,5708 rad (−90°) viaja como `BF F9 21 FF 2E 48 E8 A7`; leído en el orden del PC (little-endian) da **$-1{,}9 \cdot 10^{-116}$**.

4. Primaria o tiempo real: ¿cuál para cada uso?

Primaria: 10 mensajes/s, poco tráfico, incluye errores y acepta órdenes. **Tiempo real:** ~500 paquetes/s de estructura fija, menos retraso y unas 50 veces más tráfico. Para un **panel de supervisión**, la primaria. Para **registrar un movimiento rápido**, la de tiempo real (un dato cada 2 ms).

5. ¿Por qué con NAT no se llega a URSim y en modo puente sí? ¿Quién ve la MAC de la VM?

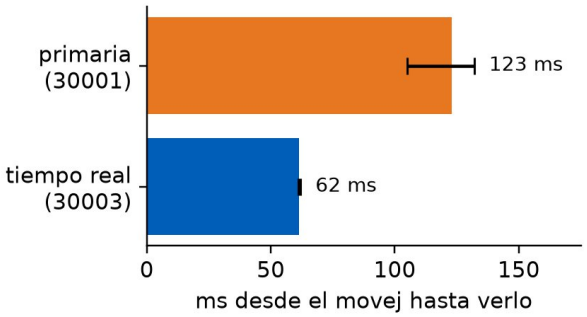
Con NAT la máquina virtual queda escondida tras una red privada de VirtualBox: puede salir, pero desde el PC no hay una IP a la que conectarse (habría que redirigir puertos). En **modo puente** es un equipo más de la red del laboratorio, con su propia IP (10.172.18.67). Su MAC la ven el **switch** y los equipos del mismo segmento, como nuestro PC en su tabla ARP.

6. Si se corta la red 5 s, ¿qué hace el programa? ¿Y TCP?

TCP no pierde datos: el robot los guarda y los reenvía; al volver la red llegan de golpe y atrasados (en un corte de 1,6 s llegaron 12 estados seguidos). **El programa** espera como máximo 5 s sin datos: si se superan, da la conexión por perdida y reconecta a los 2 s con datos frescos. Se probó cortando la red con el cortafuegos (para TCP es lo mismo que quitar el cable).

7. (N) Retraso entre URSim y el robot simulado con cada interfaz.

Medido desde el `movej` hasta ver cambiar el ángulo: **62 ms** con tiempo real y **123 ms** con la primaria. Unos 60 ms son lo que tarda el robot en arrancar el programa; el resto es el periodo de publicación (2 ms frente a 100 ms). Se suman hasta 33 ms del dibujo a 30 fotogramas/s. Depende del periodo de cada interfaz, del ritmo de dibujo, de la red y de la carga del PC y de la VM.



8. (N) ¿Qué pasaría sin Lock?

El dibujo podría leer el estado mientras el hilo lo escribe y mezclar datos de dos instantes, o fallar al copiar el diccionario justo cuando otro hilo añade un robot (`dictionary changed size during iteration`). Son fallos raros e imprevisibles; el Lock los evita.

9. (S) ¿Por qué no basta con enviar un movej para saber que el robot ha llegado?

Porque el robot no responde: la orden puede rechazarse (sintaxis, modo local), interrumpirse (parada de protección) o tardar. Se confirma con su estado: `isProgramRunning` pasa a verdadero y luego a falso, `q_actual` queda a menos de 0,01 rad del objetivo y no llegan errores (mensajes tipo 20) ni `isProtectiveStopped`. Si no arranca en 2 s, la orden se da por rechazada.

10. (S) Riesgo de que cualquiera pueda enviar programas al 30001 y dos medidas.

No hay contraseña ni cifrado: cualquier equipo de la red podría mover el robot con personas cerca, parar la producción o dañar el equipo. **En el robot:** desactivar o restringir en PolyScope los servicios entrantes (30001–30003) y dejar para supervisión solo interfaces de lectura como la 30011. **En la red:** poner los robots en una VLAN propia tras un cortafuegos que solo deje conectar a los puertos de control desde la IP autorizada (zonas y conductos de IEC 62443).